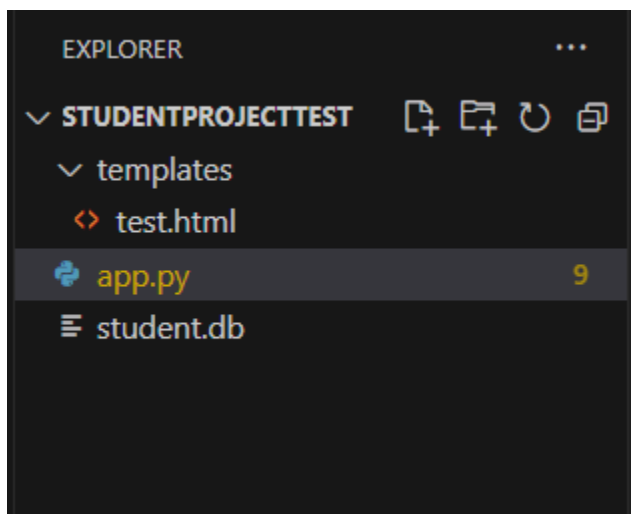


Backend Development - FLASK

Flask is a microframework for small to medium-sized applications. It adheres to the ModelViewTemplate (MVT) architectural pattern and provides only the fundamental functionality for developing web applications.

Reference: [Installation — Flask Documentation \(3.1.x\)](#)

1. Download python: [Download Python](#)
2. Download any editor: [Download Visual Studio Code - Mac, Linux, Windows](#)
3. Run this command in the terminal: `pip -m install Flask`
4. Project folder structure:



5. Command to run this code: `python app.py`
6. We will create a dummy API first.

app.py

```
from flask import Flask, request, jsonify
```

```
app = Flask(__name__)
```

```

@app.route('/test', methods=['GET', 'POST'])

def test():

    if request.method == 'GET':

        return jsonify({"response": "Get Request Called"})

    elif request.method == "POST":

        req_json = request.json

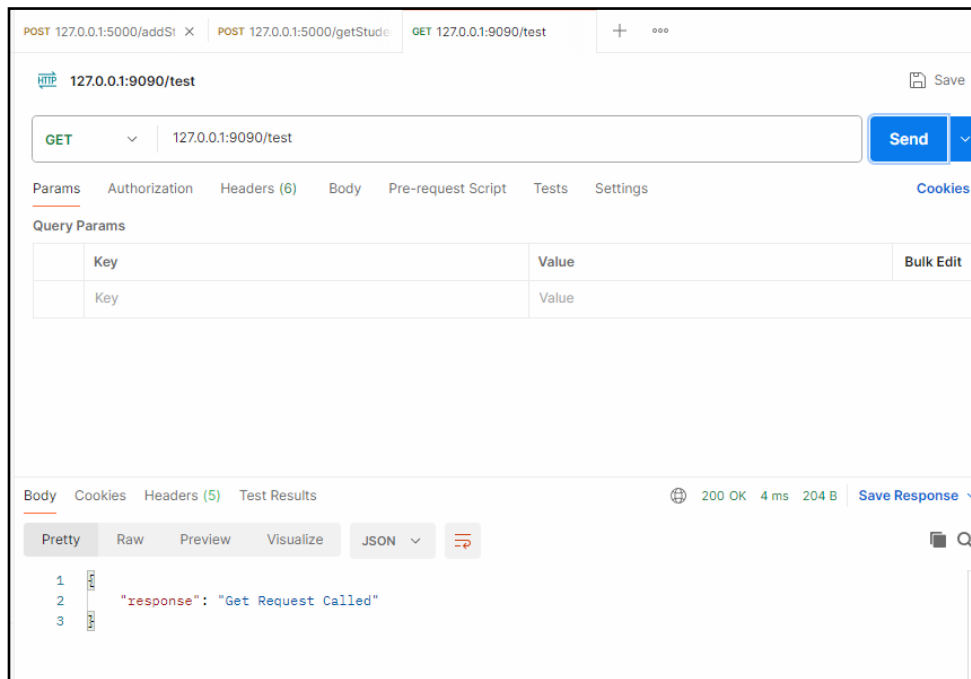
        name = req_json['name']

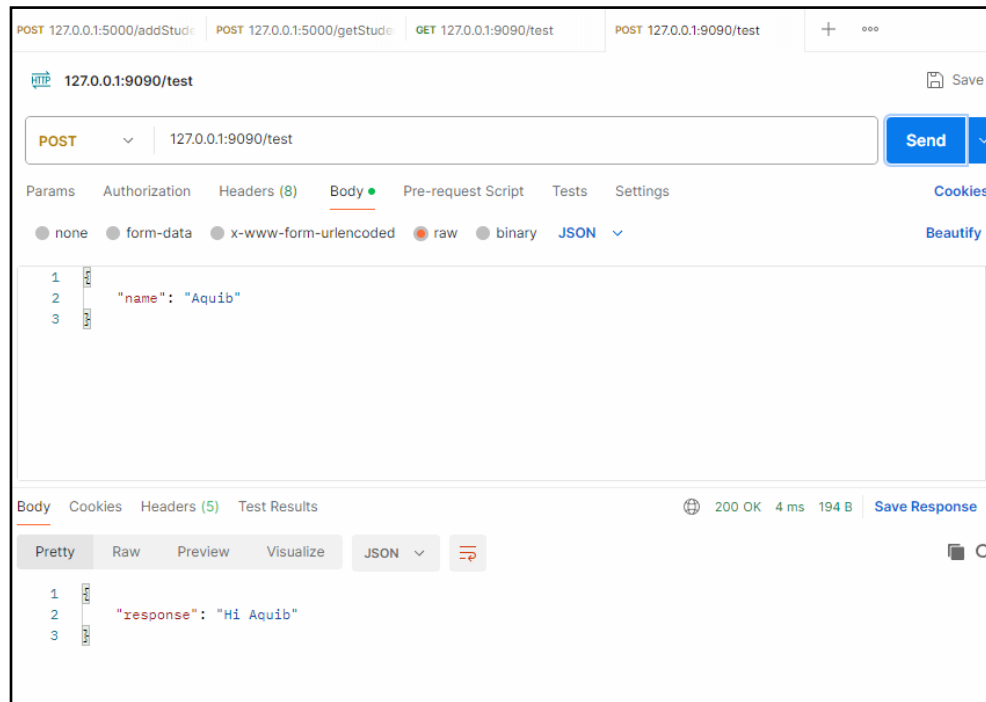
        return jsonify({"response": "Hi " + name})

if __name__ == '__main__':

    app.run(debug=True, port=9090)

```





7. Now, we will add database connection and model class

app.py

```
import sqlite3
from flask import *

import json
import uuid

class Student:

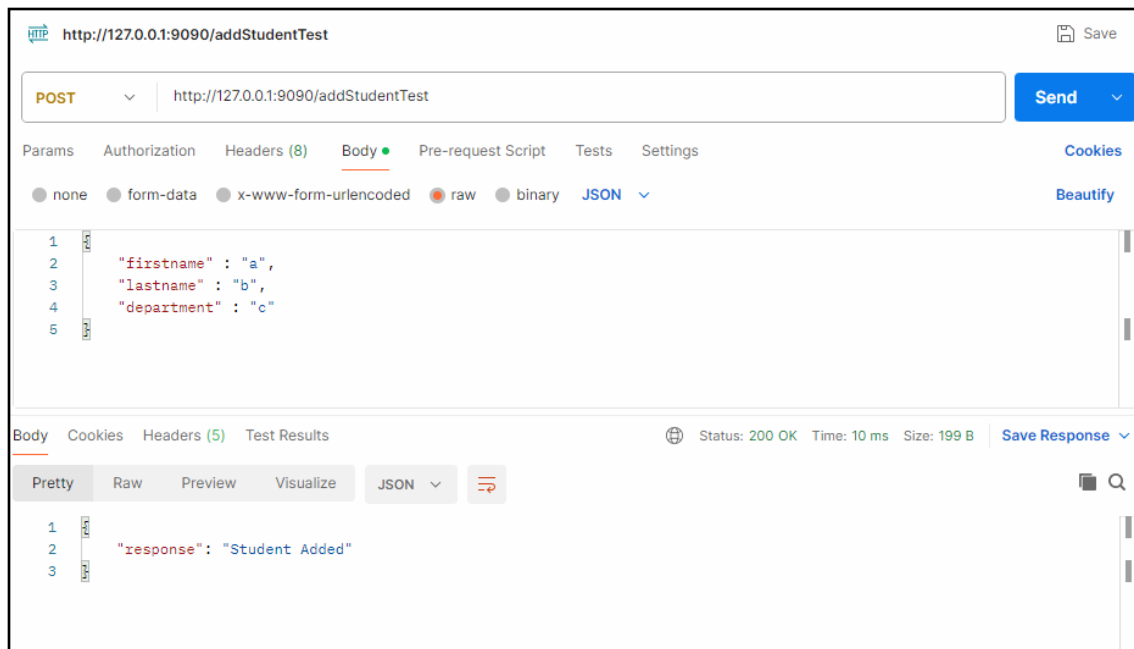
    def __init__(self, firstname, lastname, department):
        self.id = uuid.uuid4().hex
        self.firstname = firstname
        self.lastname = lastname
        self.department = department

app = Flask(__name__)
```

```
def connect_db():
    c = sqlite3.connect("student.db").cursor()
    c.execute("CREATE TABLE IF NOT EXISTS STUDENTS("
              "id TEXT, firstname TEXT, lastname TEXT, department TEXT)")
    c.connection.close()
```

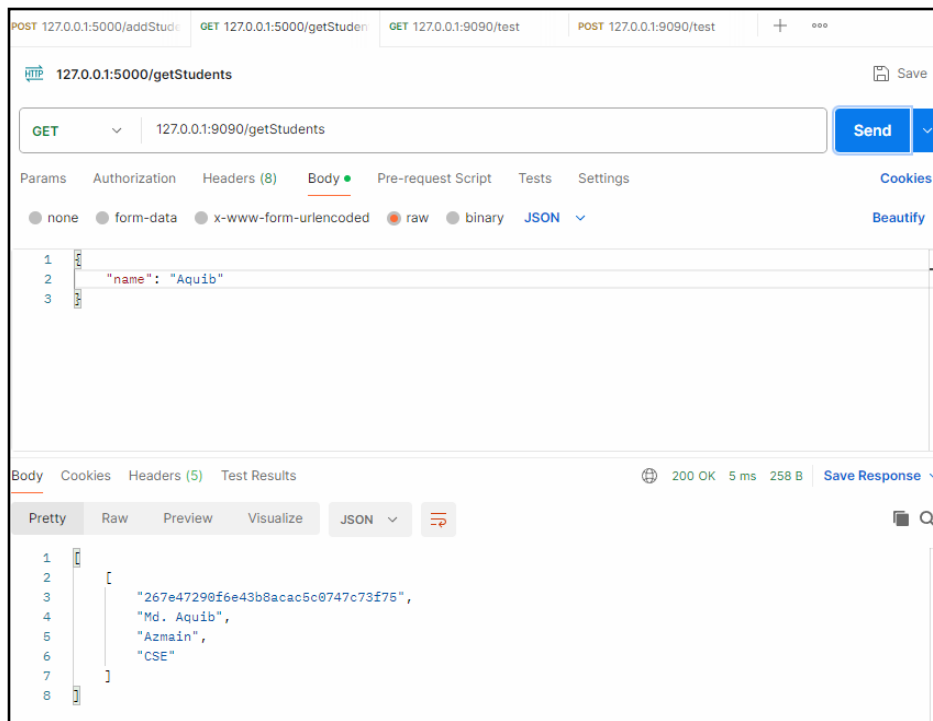
```
@app.route('/', methods=['GET'])
def index():
    connect_db()
    data = getStudents()
    return render_template('test.html', data = data)
```

```
@app.route('/addStudentTest', methods=['POST'])
def addStudentTest():
    db = sqlite3.connect("student.db")
    c = db.cursor()
    req_json = request.json
    student = Student(req_json['firstname'], req_json['lastname'], req_json['department'])
    c.execute("INSERT INTO STUDENTS VALUES(?,?,?,?)",
              (student.id, student.firstname, student.lastname,
               student.department))
    db.commit()
    return jsonify({"response": "Student Added"})
```



```
@app.route('/addStudent', methods=['POST', 'GET'])
def addStudent():
    db = sqlite3.connect("student.db")
    c = db.cursor()
    student = Student(request.form["firstname"],
                      request.form["lastname"],
                      request.form["department"])
    print(student)
    c.execute("INSERT INTO STUDENTS VALUES(?,?,?,?)",
              (student.id, student.firstname, student.lastname, student.department))
    db.commit()
    data = getStudents()
    return render_template('test.html', data = data)

@app.route('/getStudents', methods=['GET'])
def getStudents():
    c = sqlite3.connect("student.db").cursor()
    c.execute("SELECT * FROM STUDENTS")
    data = c.fetchall()
    return data
```



```
@app.route('/deleteStudent', methods=['POST'])
def deleteStudent():
    student_id = request.form["id"]
    db = sqlite3.connect("student.db")
    c = db.cursor()
    c.execute("DELETE FROM STUDENTS WHERE id=?", (student_id,))
    db.commit()
    data = getStudents()
    return render_template('test.html', data = data)
```

```
if __name__ == '__main__':
```

```
    app.run(debug=True, port=9090)
```

Use below delete API for postman test. To render with HTML template, edit 46 no line as
`<form action="{{ url_for('deleteStudent', student_id=item[0]) }}" method="post">`

```
@app.route('/deleteStudent/<student_id>', methods=['POST'])
def deleteStudent(student_id):
    db = sqlite3.connect("student.db")
    c = db.cursor()

    c.execute("DELETE FROM STUDENTS WHERE id=?", (student_id,))

    db.commit()

    data = getStudents()

    # return render_template('test.html', data = data)

    return jsonify({"response": "Student Deleted", "data": data})
```

test.html

```
<!DOCTYPE html>
<html>
<title>Test App</title>
<meta name="viewport" content="width=device-width, initial-scale=1">
<link rel="stylesheet" href="https://www.w3schools.com/w3css/4/w3.css">
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-
awesome.min.css">

<body>

    <header class="w3-container w3-teal">
        <h1>Flask Example</h1>
    </header>

    <div style="margin: auto; width: 60%; padding-top: 10px">
        <div class="w3-container w3-margin-top">

            <form action="{{ url_for('addStudent') }}" method="post" class="w3-container w3-card-
4">
```

```

    <p>
      <input class="w3-input" type="text" name="firstname" required>
      <label>First Name</label>
    </p>
    <p>
      <input class="w3-input" type="text" name="lastname" required>
      <label>Last Name</label>
    </p>
    <input class="w3-input" type="text" name="department" required>
    <label>Department</label></p>

    <p>
      <button class="w3-button w3-section w3-teal w3-ripple"> Submit </button>
    </p>

  </form>

</div>
</div>

<div style="margin: auto; width: 60%; padding-top:20px">
  <div class="w3-container">
    <h2>Student List</h2>
    <ul class="w3-ul w3-card-4">
      {% for item in data %}
      <li class="w3-bar">
        <span class="w3-white w3-right">
          <form action="{{ url_for('deleteStudent') }}" method="post">
            <input type="hidden" value="{{ item[0] }}" name="id" />
            <input class="w3-button w3-section w3-teal w3-ripple" type="submit"
value="Remove"/>
          </form>
        </span>
        
        <div class="w3-bar-item">
          <span class="w3-large"> {{ item[1] }} {{ item[2] }} </span><br>
          <span> {{ item[3] }} </span>
        </div>
      </li>

```

```
        {% endfor %}
    </ul>
</div>
</div>
</body>

</html>
```

We can view the student.db file using: [SQLite Viewer](#)

The **commit()** method is one among the various methods in Python which is used to make the database transactions. The commit() method is used to confirm the changes made by the user to the database. Whenever any change is made to the database using update or any other statements, it is necessary to commit the changes. If we don't use the commit() method after making any changes to the database, the database will not be updated and changes will not be reflected.

sqlite3.Cursor class is an instance using which you can invoke methods that execute SQLite statements, fetch data from the result sets of the queries. You can create a Cursor object using the cursor() method of the Connection object/class.

External Api with postman

Open-Meteo Weather API Demo

1. Current Weather

Endpoint: GET <https://api.open-meteo.com/v1/forecast>

Params:

- latitude
- longitude
- current_weather=true

Requests included:

- Dhaka: latitude=23.8103, longitude=90.4125
- London: latitude=51.5072, longitude=-0.1276
- Tokyo: latitude=35.6764, longitude=139.6500

GET

https://api.open-meteo.com/v1/forecast?latitude=23.8103&longitude=90.4125¤t_weather=true

2. Hourly Forecast

Endpoint: same as above

Params:

- latitude, longitude
- hourly=temperature_2m,relative_humidity_2m
- timezone=auto

GET

https://api.open-meteo.com/v1/forecast?latitude=23.8103&longitude=90.4125&hourly=temperature_2m,relative_humidity_2m&timezone=Asia/Dhaka

3. Daily Forecast Example

GET

https://api.open-meteo.com/v1/forecast?latitude=23.8103&longitude=90.4125&daily=temperature_2m_max,temperature_2m_min,sunrise,sunset&timezone=Asia/Dhaka

